
第4章 机器学习

4.1 机器学习算法分类

机器学习算法可以根据训练数据是否带有标签、以及训练方式的不同，大致分为三大类：监督学习（Supervised Learning）、无监督学习（Unsupervised Learning）和半监督学习（Semi-Supervised Learning），此外还有近年来广泛应用的迁移学习（Transfer Learning）。本节将分别介绍这几类学习方法的基本概念、特点及典型应用场景。

4.1.1 监督学习与常见应用场景

监督学习是指在训练过程中，模型同时接收输入数据（特征）和对应的正确输出（标签），通过学习输入与输出之间的映射关系，从而对新的、未知的数据进行预测。

1. 基本原理

在监督学习中，训练数据集通常表示为：

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

其中， x_i 是输入样本（通常为一个向量）， y_i 是对应的真实标签（可以是类别或数值）。模型的目标是学习一个函数 $f: X \rightarrow Y$ ，使得对于新输入 x ，能够预测出尽可能准确的 $y = f(x)$ 。

2. 分类

根据预测目标的不同，监督学习又可以分为两大子类：

分类（Classification）：当标签 y 是离散的类别时，如“猫/狗”、“垃圾邮件/正常邮件”。常见算法：

逻辑回归（Logistic Regression）

支持向量机（Support Vector Machine, SVM）

决策树（Decision Tree）

随机森林（Random Forest）

K 近邻（K-Nearest Neighbors, KNN）

神经网络（Neural Networks）

回归（Regression）：当标签 y 是连续值时，如房价、温度、销售额等。常见算法：

线性回归（Linear Regression）

岭回归（Ridge Regression）

Lasso 回归

支持向量回归（SVR）

神经网络回归模型

3. 常见应用场景

图像分类（如手写数字识别、物体识别）

垃圾邮件检测

医疗诊断（如疾病预测、医学图像分析）

金融风控（如信用评分、欺诈检测）

房价预测、股票价格趋势预测

语音识别中的文本转录（带标注数据）

4.1.2 无监督学习与常见应用场景

无监督学习是指训练数据中仅包含输入样本 x_i ，而没有对应的标签 y_i 。模型需要从数据本身的结构、分布或模式中学习信息，常用于发现数据中的隐藏规律或进行数据降维与分组。

1. 基本原理

无监督学习的训练数据通常为：

$$D = \{x_1, x_2, \dots, x_n\}$$

模型尝试找出这些数据中的某种结构，比如聚类、分布、异常等，而不依赖于任何"正确答案"。

2. 分类

聚类（Clustering）：将数据划分为若干组（簇），使得同一组内数据相似度高，不同组之间差异大。常见算法：

K 均值聚类（K-Means Clustering）

层次聚类（Hierarchical Clustering）

DBSCAN（基于密度的聚类）

高斯混合模型（Gaussian Mixture Model, GMM）

降维（Dimensionality Reduction）：减少数据的特征数量，同时尽量保留重要信息。常见算法：

主成分分析（Principal Component Analysis, PCA）

t-SNE（t-Distributed Stochastic Neighbor Embedding）

自编码器（Autoencoders，属于深度学习方法）

关联规则挖掘（Association Rule Learning）：发现数据项之间的有趣关系，如"购买了 A 的人也倾向于购买 B"。典型算法：

Apriori 算法

FP-Growth 算法

异常检测（Anomaly Detection）：识别与大多数数据显著不同的样本，如网络入侵检测、设备故障预警。

3. 常见应用场景

用户分群（如市场细分、客户群体分析）

数据压缩与可视化（如通过 PCA 或 t-SNE 对高维数据进行降维展示）

异常检测（如信用卡欺诈、工业设备异常）

推荐系统中的用户行为聚类

文本主题建模（如 LDA，虽属于生成模型但也常在无监督框架下使用）

4.1.3 半监督学习与迁移学习

随着数据采集成本降低，实际应用中往往能获得大量未标注数据和少量标注数据，这就催生了半监督学习与迁移学习方法，它们在一定程度上弥补了监督学习对大量标注数据的依赖。

1. 半监督学习

半监督学习是介于监督学习和无监督学习之间的一种学习方式，其训练数据中只有一小部分带有标签，大部分数据没有标签。该方法旨在利用大量未标注数据中的信息来辅助提升模型的泛化能力。

基本思想：半监督学习假设数据分布本身包含可被挖掘的结构信息，即使没有显式的标签。通过结合少量有标签数据和大量无标签数据，模型可以学到更鲁棒的特征表达和决策边界。

常见方法：

自训练（Self-training）：先用有标签数据训练一个初始模型，然后对无标签数据进行预测，将高置信度的预测结果加入训练集，迭代训练。

协同训练（Co-training）：多个视图（feature sets）下的模型相互教学，互为补充。

生成式模型方法（如基于图的方法、拉普拉斯正则化等）

半监督深度学习（如在神经网络训练中引入未标注样本的 consistency regularization）

应用场景：

医学图像分析（标注昂贵，但未标注数据丰富）

语音识别、自然语言处理中的大规模语料预训练

工业质检（有少量缺陷样本，但大量正常样本未标注）

2. 迁移学习

迁移学习是指将在一个任务（源任务）上学习到的知识（如特征、模型参数等）迁移到另一个相关但不同的任务（目标任务）上，尤其适用于目标任务标注数据较少、但源任务数据丰富的情形。

基本思想：迁移学习的核心理念是"知识复用"。通过利用源领域中学到的通用特征表示，可以加速目标领域的学习过程，提高模型性能，尤其在数据稀缺时效果显著。

常见策略：

特征迁移（Feature Transfer）：将在源任务上学到的特征表示直接应用于目标任务，如使用预训练的 CNN 底层特征。

模型微调 (Fine-tuning)：在预训练模型（如 BERT、ResNet 等）的基础上，针对目标任务进行少量参数调整。

参数共享 / 多任务学习：多个任务共享一部分模型参数，互相促进。

跨域适应 (Domain Adaptation)：源域与目标域数据分布不同，但希望模型能在目标域上表现良好。

应用场景：

自然语言处理中的预训练语言模型（如 GPT、BERT 等，在大规模语料上预训练后微调至具体任务）

计算机视觉中的图像分类、目标检测（如利用 ImageNet 预训练模型）

医疗、法律等专业领域，标注数据稀缺但可借助通用模型迁移

跨语言、跨模态任务（如将图像知识迁移到文本描述生成）

3. 总结对比

类型	是否有标签	数据要求	典型应用
监督学习	有	大量标注数据	分类、回归、预测
无监督学习	无	无标签数据	聚类、降维、异常检测
半监督学习	部分有	大量无标签 + 少量有标签	标注成本高的场景
迁移学习	通常有（源任务）	源任务数据丰富，目标任务数据少	预训练 + 微调、跨领域任务

4.2 机器学习中的模型评估和选择

在机器学习项目中，构建模型的最终目的是使其在未知数据上具有良好的泛化能力。本节将围绕数据集划分、评估指标、交叉验证与模型选择、以及过拟合与欠拟合的识别与应对策略展开详细阐述。

4.2.1 数据集划分：训练集、验证集与测试集

为了评估模型的泛化能力，通常需要将收集到的数据集划分为不同的子集，分别用于模型的训练、参数调优和最终评估。

1. 基本划分方式

训练集 (Training Set)：用于训练模型参数，即通过最小化损失函数来学习输入与输出之间的映射关系。

验证集 (Validation Set)：用于模型选择、超参数调优和评估不同算法或模型结构的性能，但不用于最终评价。

测试集 (Test Set)：用于模拟模型在"完全未知"的数据上的表现，是评估模型泛化能力的最终标准。

2. 常见划分比例

小规模数据（几万条以内）：训练集 70%、验证集 15%、测试集 15%

中大规模数据：训练集 80%、验证集 10%、测试集 10% 或训练集 90%、测试集 10%（无验证集时）

3. 注意点

测试集仅在最终评估时使用一次，避免"偷看"测试集导致评估失真。

若数据量较小，可以采用交叉验证代替固定的验证集。

在深度学习任务中，有时会使用"训练集 + 验证集"合并后再划分，或者采用 k 折交叉验证。

4.2.2 常见评估指标（准确率、精确率、召回率、F1 值等）

模型评估的核心是选择合适的指标来衡量其性能。不同的任务（如分类、回归、排序等）需采用不同的评估指标。

1. 混淆矩阵（Confusion Matrix）

对于二分类问题，混淆矩阵定义如下：

	预测为正类	预测为负类
实际为正类	真正例（True Positive, TP）	假负例（False Negative, FN）
实际为负类	假正例（False Positive, FP）	真负例（True Negative, TN）

2. 准确率（Accuracy）

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

表示模型预测正确的样本占总样本的比例。但在类别不平衡时可能具有误导性。

3. 精确率（Precision）

$$\text{Precision} = TP / (TP + FP)$$

表示所有被模型预测为正类的样本中，真正为正类的比例。关注"预测可靠性"。

4. 召回率（Recall）

$$\text{Recall} = TP / (TP + FN)$$

表示所有真正为正类的样本中，被模型正确预测出来的比例。关注"覆盖率"。

5. F1 值（F1-Score）

$$F1 = 2 \cdot (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

是精确率和召回率的调和平均数，用于综合评估两者表现，特别适用于类别不平衡或两者都重要的场景。

6. 特异度（Specificity）

$$\text{Specificity} = TN / (TN + FP)$$

表示负类样本中被正确预测的比例。

7. 回归任务常用指标

均方误差 (MSE):

$$MSE = (1/n) \sum_i (y_i - \hat{y}_i)^2$$

均方根误差 (RMSE):

$$RMSE = \sqrt{MSE}$$

平均绝对误差 (MAE):

$$MAE = (1/n) \sum_i |y_i - \hat{y}_i|$$

决定系数 (R^2 Score):

$$R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$$

表示模型解释目标变量方差的比例，越接近 1 越好。

4.2.3 交叉验证与模型选择方法

为了更稳健地评估模型性能，避免因单次数据划分带来的偶然性，常采用交叉验证 (Cross Validation) 方法。

1. k 折交叉验证 (k-Fold Cross Validation)

将训练集随机且均匀地划分为 k 个大小相似的子集 (折)，依次使用其中的 k-1 折作为训练数据，剩下 1 折作为验证数据，重复 k 次，最后取 k 次评估结果的平均值作为模型性能的估计。

最常用的是 k=5 或 k=10。

优点：充分利用数据，减小因数据划分带来的偏差。

缺点：计算开销较大。

2. 留一法交叉验证 (LOOCV)

每次只留一个样本作为验证集，其余全部作为训练集，重复 n 次 (n 为样本总数)。是 k 折交叉验证在 k=n 时的特殊情况。适用于极小型数据集，但计算成本极高。

3. 模型选择方法

在验证集或通过交叉验证得到各模型的性能指标 (如准确率、F1、MSE 等)。

选择在验证集上表现最优的模型或参数组合。

最终在独立的测试集上进行一次性的最终评估。

4. 网格搜索与随机搜索

网格搜索 (Grid Search): 穷举给定参数范围内的所有组合，计算量大但全面。

随机搜索 (Random Search): 随机采样参数组合，更高效，常能获得接近最优解。

4.2.4 过拟合与欠拟合的识别与应对

在模型训练过程中，常常面临两种典型的问题：过拟合（Overfitting）与欠拟合（Underfitting），它们直接影响模型的泛化能力。

1. 欠拟合（Underfitting）

现象：模型在训练集和验证集上都表现不佳，无法捕捉数据的基本模式。

原因：

模型过于简单（如线性模型拟合非线性数据）

特征不足或特征工程不充分

训练时间不足或算法未收敛

解决方法：

增加模型复杂度（如使用非线性模型、更深的网络）

增强特征工程，构造更有意义的特征

延长训练时间，调整优化算法参数

2. 过拟合（Overfitting）

现象：模型在训练集上表现很好，但在验证集或测试集上表现差，即"记住"而非"学习"了数据。

原因：

模型过于复杂而训练数据不足

训练数据噪声较多

模型过度记忆训练样本中的噪声或特例

解决方法：

增加训练数据量（数据增强）

使用正则化方法（如 L1/L2 正则化、Dropout）

简化模型结构（减少参数量、层数等）

早停（Early Stopping）：在验证误差不再下降时停止训练

使用交叉验证选择更鲁棒的模型

进行模型集成（如 Bagging、Boosting）

3. 如何判断过拟合/欠拟合？

训练误差低，验证误差高 → 过拟合

训练误差高，验证误差高 → 欠拟合

两者都低 → 模型较为合适

4. 总结对比

特性	欠拟合 (Underfitting)	过拟合 (Overfitting)
训练集表现	差	好
验证/测试集表现	差	差
模型复杂度	过低	过高
解决方法	增强模型/特征	正则化/简化模型/增加数据

4.3 常见机器学习算法简介

4.3.1 线性模型：线性回归与逻辑回归

线性回归

线性回归是监督学习中最简单的回归算法，旨在建立输入特征与连续目标变量之间的线性关系模型。

数学模型：给定训练数据集 $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ ，其中 $x_i \in \mathbb{R}^d$ 是特征向量， $y_i \in \mathbb{R}$ 是连续目标值。线性回归模型假设：

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d + \varepsilon = \theta^T X + \varepsilon$$

其中 $\theta = [\theta_0, \theta_1, \dots, \theta_d]^T$ 是模型参数（包括偏置项 θ_0 ）， ε 是误差项。

参数估计：最小二乘法 模型参数通过最小化残差平方和（RSS）来估计：

$$J(\theta) = (1/2n) \sum_i (y_i - \theta^T x_i)^2$$

使用矩阵表示法，最优参数的解析解为：

$$\theta = (X^T X)^{-1} X^T y$$

梯度下降算法：参数更新规则为：

$$\theta_j := \theta_j - \alpha \cdot (1/n) \sum_i ((\theta^T x_i) - y_i) x_{ij}$$

其中 α 是学习率，控制更新步长。

正则化：岭回归与 Lasso：

岭回归（L2 正则化）： $J(\theta) = \text{RSS} + \lambda \sum_j \theta_j^2$

Lasso 回归（L1 正则化）： $J(\theta) = \text{RSS} + \lambda \sum_j |\theta_j|$

Lasso 具有特征选择能力，能将不重要的特征系数压缩为 0。

逻辑回归

逻辑回归实为分类算法，通过逻辑函数将线性组合映射到概率空间。

数学模型：

$$P(y = 1 | x) = \sigma(\theta^T x) = 1 / (1 + e^{-(\theta^T x)})$$

其中 $\sigma(z) = 1/(1 + e^{-z})$ 是 sigmoid 函数，将线性组合压缩到 (0,1) 区间。

多分类扩展（Softmax 回归）：

$$P(y = k | x) = e^{(\theta_k^T x)} / \sum_j e^{(\theta_j^T x)}$$

4.3.2 基于树的方法：决策树与随机森林

基于树的方法以其直观性、非参数特性和处理混合类型数据的能力而备受青睐。

决策树

决策树通过一系列 if-then 规则对数据进行递归划分，模拟人类决策过程。

分裂标准：

信息增益 (ID3)：

$$\text{Entropy}(D) = - \sum_i p_i \log_2 p_i$$

$$\text{InfoGain}(D, A) = \text{Entropy}(D) - \sum_v (|D_v|/|D|) \text{Entropy}(D_v)$$

信息增益比 (C4.5)：

$$\text{GainRatio}(D, A) = \text{InfoGain}(D, A) / \text{SplitInfo}(D, A)$$

基尼指数 (CART)：

$$\text{Gini}(D) = 1 - \sum_i p_i^2$$

剪枝策略：

预剪枝：在生长过程中提前终止（如设定最大深度、最小样本数）

后剪枝：先生成完整树，再自底向上剪枝（如代价复杂度剪枝）

随机森林

随机森林是决策树的集成方法，通过组合多个弱学习器构建强学习器。随机森林属于 Bagging 家族，核心思想：

从训练集有放回抽样生成多个 Bootstrap 样本集

对每个 Bootstrap 集训练决策树

综合所有树的预测结果（投票或平均）

预测方式：

分类：多数投票 $\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_B(x)\}$

回归：平均值 $\hat{y} = (1/B) \sum_{\beta} T_{\beta}(x)$

4.3.3 支持向量机 (SVM)

SVM 的核心思想是寻找一个最优超平面，使得两类样本之间的间隔最大化。

硬间隔 SVM：

$$\min_{\{w, b\}} (1/2) \|w\|^2 \quad \text{s.t. } y_i(w^T x_i + b) \geq 1, i = 1, \dots, n$$

软间隔 SVM（引入松弛变量 $\xi_i \geq 0$ ）：

$$\min_{\{w, b, \xi\}} (1/2) \|w\|^2 + C \sum_i \xi_i \quad \text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i$$

常用核函数：

线性核： $\kappa(x_i, x_j) = x_i^T x_j$

多项式核: $\kappa(x_i, x_j) = (\gamma x_i^T x_j + r)^d$

RBF 核 (高斯核): $\kappa(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$

Sigmoid 核: $\kappa(x_i, x_j) = \tanh(\gamma x_i^T x_j + r)$

4.3.4 聚类算法: K-Means 与层次聚类

K-Means 聚类: 目标是将 n 个样本划分到 k 个簇, 使每个样本到其所属簇中心的距离平方和最小:

$$J = \sum_i \sum_{x \in C_i} \|x - \mu_i\|^2$$

算法流程:

初始化: 随机选择 k 个样本作为初始簇中心

分配步骤: 将每个样本分配到最近的簇中心

更新步骤: 重新计算每个簇的质心 $\mu_i = (1/|C_i|) \sum_{x \in C_i} x$

收敛判断: 重复直到簇分配不再变化

层次聚类: 通过构建树形结构 (dendrogram) 展示数据层次关系, 不需预设簇数。常用连接准则:

单连接: $\text{dmin}(C_i, C_j) = \min_{\{x \in C_i, y \in C_j\}} d(x, y)$

全连接: $\text{dmax}(C_i, C_j) = \max_{\{x \in C_i, y \in C_j\}} d(x, y)$

平均连接: $\text{davg}(C_i, C_j) = (1/|C_i||C_j|) \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$

Ward 方法: 合并使总离差平方和增加最小的簇

特性	K-Means	层次聚类
聚类形状	球形簇	任意形状 (依赖连接准则)
簇数确定	需预设 k 值	通过树状图选择
计算复杂度	$O(nktd)$	$O(n^3)$ 或 $O(n^2 \log n)$
可扩展性	适合大规模数据	适合中小规模数据
结果稳定性	受初始值影响	确定性结果
最佳适用场景	大规模、球形簇、已知 k 值	小规模、探索性分析、需层次关系

4.3.5 降维方法: 主成分分析 (PCA)

主成分分析是最经典和广泛使用的线性降维技术, 旨在通过正交变换将相关变量转化为不相关的主成分, 同时保留数据主要变异信息。

1. 算法推导: 三种等价视角

最大方差视角: 第一主成分是使投影方差最大的方向:

$$\max_{\{w_1\}} (1/n) \sum_i (w_1^T x_i)^2 = w_1^T S w_1 \quad \text{s.t.} \quad \|w_1\| = 1$$

最小重构误差视角：

$$\min_W \|X - XWW^T\|_F^2 \quad \text{s.t. } W^TW = I$$

特征分解视角：主成分是协方差矩阵的特征向量，按特征值降序排列：

$$Sw_j = \lambda_j w_j, \quad j = 1, \dots, p$$

2. 算法步骤

数据标准化：将每个特征标准化为均值为 0，标准差为 1： $x'_{ij} = (x_{ij} - \mu_j) / \sigma_j$

计算协方差矩阵： $S = (1/(n-1)) X^TX$

特征分解： $SW = W\Lambda$ ，其中 $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$

选择主成分：按特征值大小排序，选择前 k 个特征向量

投影变换： $T = XW_k$ ，得到 k 维主成分得分

3. 主成分选择策略

方差解释率：累计解释方差比例：

$$R_k = (\sum_{j=1}^k \lambda_j) / (\sum_{i=1}^p \lambda_i)$$

通常选择使累计解释率超过阈值（如 80% 或 90%）的最小 k 。

4.4 本章实践

本节通过完整的实践案例来加深理解。实践部分包括环境准备、数据集使用、模型训练、评估与结果可视化等环节。

4.4.1 环境准备与数据集介绍

推荐使用 Python 语言及其主流的科学计算与机器学习库：

Python 3.8+：编程语言

NumPy：数值计算

Pandas：数据处理与清洗

Matplotlib / Seaborn：可视化工具

scikit-learn：机器学习模型库

在终端执行以下命令完成环境配置：

```
pip install numpy pandas matplotlib seaborn scikit-learn
```

数据集选择了波士顿房价数据集（Boston Housing Dataset）和鸢尾花数据集（Iris Dataset），前者用于回归预测，后者用于分类任务。

4.4.2 使用线性回归完成房价预测

房价预测是回归问题的经典应用。

1. 数据加载与预处理

```
from sklearn.datasets import load_boston
import pandas as pd
import numpy as np

# 加载数据
boston = load_boston()
X = pd.DataFrame(boston.data, columns=boston.feature_names)
y = pd.Series(boston.target, name="PRICE")

# 划分训练集和测试集
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

2. 训练模型

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
print("模型截距:", model.intercept_)
print("模型系数:", model.coef_)
```

3. 模型预测与评估

```
from sklearn.metrics import mean_squared_error, r2_score

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("均方误差 (MSE):", mse)
print("决定系数 (R²):", r2)
```

4.4.3 使用决策树进行分类任务

1. 数据加载与划分

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

2. 训练与评估决策树

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

clf = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print("分类准确率:", accuracy_score(y_test, y_pred))
print("详细分类报告:\n", classification_report(y_test, y_pred))
```

4.5 本章实践（扩展版）

4.5.1 Python 机器学习环境配置

```
# 创建虚拟环境
conda create -n ml-practice python=3.9
conda activate ml-practice

# 安装核心库
pip install numpy pandas matplotlib seaborn
pip install scikit-learn
pip install jupyter
```

4.5.2 使用线性回归完成房价预测（详细版）

1. 数据探索与预处理

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# 数据基本信息探索
print("=== 数据基本统计 ===")
print(boston_df.describe())

# 特征标准化（重要步骤！）
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

4.5.3 使用决策树进行分类任务（详细版）

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix
```

```
# 创建决策树分类器（限制树深度防止过拟合）
```

```
dt_model = DecisionTreeClassifier(
    max_depth=3,          # 最大深度
    min_samples_split=5,  # 内部节点再划分所需最小样本数
    min_samples_leaf=2,   # 叶节点最少样本数
    random_state=42
)
dt_model.fit(X_train_iris, y_train_iris)
```

4.5.4 模型评估与超参数调优

```
from sklearn.model_selection import GridSearchCV
```

```
# 决策树超参数调优
```

```
param_grid = {
    'max_depth': [2, 3, 4, 5, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

```
dt_tuned = DecisionTreeClassifier(random_state=42)
grid_search = GridSearchCV(dt_tuned, param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train_iris, y_train_iris)
print("最佳超参数:", grid_search.best_params_)
print(f"最佳交叉验证准确率: {grid_search.best_score_:.3f}")
```

第5章 深度学习

5.1 深度学习环境配置（CUDA）及线上环境使用

在开始深度学习的实验之前，配置一个合适的环境是至关重要的步骤。环境的选择与搭建直接决定了模型训练是否能够高效进行。本节将详细介绍 CUDA 与 GPU 加速的基本概念，本地深度学习环境的搭建方法，以及如何使用华为云与数字实验平台进行远程实验。同时，我们还会学习 Jupyter Notebook 这一常用的远程开发环境。

5.1.1 CUDA 与 GPU 加速的基本概念

深度学习的核心是大规模的矩阵运算，尤其是反向传播中的张量计算。传统 CPU 在并行计算上的效率有限，而 GPU（Graphics Processing Unit）凭借数千个流处理器，可以实现大规模的并行计算，从而显著加速神经网络训练。

1. GPU 的并行计算特性

CPU（中央处理器）适合处理复杂的逻辑控制和少量并行任务，而 GPU 则专门设计用于执行大规模相同的运算任务。现代 GPU 通常包含数千个核心，可以同时进行浮点运算。这种设计使其在矩阵乘法、卷积等任务上拥有数量级的加速效果。

2. CUDA 的作用

CUDA（Compute Unified Device Architecture）是 NVIDIA 提供的一套通用并行计算架构和 API，它允许开发者通过 C/C++、Python 等语言直接调用 GPU 的计算能力。在深度学习框架（如 PyTorch、TensorFlow）中，CUDA 已经被高度封装，用户只需简单调用 `.cuda()` 或 `to(device)` 即可将张量运算移到 GPU 上。

3. GPU 加速效果示例

例如，我们比较在 CPU 和 GPU 上进行大矩阵乘法的速度：

```
import torch
import time

# 创建一个 10000×10000 的随机矩阵
size = 10000
a = torch.rand(size, size)
b = torch.rand(size, size)

# 在 CPU 上计算
start = time.time()
c = torch.mm(a, b)
end = time.time()
```

```
print("CPU 时间:", end - start)

# 在 GPU 上计算
a_gpu = a.cuda()
b_gpu = b.cuda()
torch.cuda.synchronize() # 等待 GPU 空闲
start = time.time()
c_gpu = torch.mm(a_gpu, b_gpu)
torch.cuda.synchronize()
end = time.time()
print("GPU 时间:", end - start)
```

运行后你会发现，GPU 计算速度通常比 CPU 快 10 倍甚至 100 倍，尤其在模型规模增大时优势更明显。

5.1.2 本地环境安装与配置（Python、CUDA、cuDNN、PyTorch）

1. Python 安装

推荐使用 Anaconda 管理 Python 环境，因为它可以避免依赖冲突。安装步骤如下：

下载 Anaconda：访问 <https://www.anaconda.com/download>，选择适合自己系统的版本。

安装并创建虚拟环境：

```
conda create -n dl python=3.9
conda activate dl
```

2. CUDA 安装

首先确认自己的 GPU 型号（例如 NVIDIA GeForce RTX 3080）。

确认驱动版本：nvidia-smi 命令可以查看驱动版本和 GPU 状态。

从 NVIDIA 官网下载对应版本的 CUDA Toolkit。

安装完成后，检查是否成功：

```
nvcc -V
```

3. cuDNN 安装

cuDNN 是 NVIDIA 提供的深度神经网络计算库，对卷积操作做了高度优化。

访问 NVIDIA cuDNN 官网，下载与 CUDA 版本匹配的 cuDNN。

将下载的文件解压，并复制到 CUDA 安装路径下的对应文件夹，例如：`/usr/local/cuda/include` 与 `/usr/local/cuda/lib64`

验证安装是否成功：在 PyTorch 中运行简单的神经网络训练，若能正常调用 GPU，则说明 cuDNN 安装正确。

4. PyTorch 安装

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

安装完成后，可以通过以下代码测试 GPU 是否可用：

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

5.1.3 线上环境（华为云/数字实验平台）的注册与使用

1. 华为云 ModelArts

注册华为云账号，进入 ModelArts 控制台。

创建 Notebook 任务，选择 GPU 实例（如 V100、A100）。

上传数据集，或者通过 OBS（对象存储服务）挂载数据。

在 Notebook 界面运行 PyTorch 代码，训练深度学习模型。

2. 数字实验平台

登录实验平台，申请 GPU 资源。

在远程容器或虚拟机中加载环境（通常已预装 CUDA 与 PyTorch）。

使用 JupyterLab 或远程 VSCode 进行代码开发。

实验完成后释放资源，避免资源浪费。

5.1.4 Jupyter Notebook 与远程开发环境简介

1. Jupyter Notebook 基本使用

安装：pip install jupyterlab

启动 Notebook：在终端输入 jupyter lab

在浏览器中打开 <http://localhost:8888>

新建 Python 文件，编写并运行代码。

2. Notebook 示例：训练一个简单的神经网络

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

# 数据预处理
transform = transforms.ToTensor()
train_data = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
train_loader = torch.utils.data.DataLoader(train_data, batch_size=64, shuffle=True)

# 定义网络
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(28*28, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net().to(device)
optimizer = optim.Adam(model.parameters())

# 训练
for epoch in range(1, 3):
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = nn.CrossEntropyLoss()(output, target)
        loss.backward()
        optimizer.step()
    print(f'Epoch {epoch}, Loss: {loss.item()}')
```

5.1.5 小结

本节从理论到实践介绍了 GPU 加速的基本概念，展示了本地环境配置流程，以及如何在云端和数字实验平台上快速搭建实验环境，并结合 Jupyter Notebook 的使用方法展示了实际应用。这些技能将为后续的深度学习实验提供坚实的基础。

5.2 深度学习框架介绍（PyTorch）

PyTorch 因为其动态计算图、简洁易用的 API 设计以及与 Python 科学计算生态的高度融合，成为学术界和工业界最受欢迎的框架之一。本节将系统性地介绍 PyTorch 的基本使用方法，从张量操作到自动求导，再到模型训练流程与数据加载。

5.2.1 PyTorch 基本张量操作

PyTorch 的核心数据结构是 Tensor，它与 Numpy 的 ndarray 十分类似，但额外支持 GPU 加速和自动求导。

```
import torch
import numpy as np

# 从 Python 列表创建
a = torch.tensor([1, 2, 3], dtype=torch.float32)

# 从 Numpy 数组创建
np_array = np.array([[1, 2], [3, 4]])
b = torch.from_numpy(np_array)

# 使用内置函数
c = torch.zeros((2, 3))    # 2x3 全零张量
d = torch.ones((2, 3))     # 全一张量
e = torch.randn((2, 3))    # 服从标准正态分布

print(a.dtype, a.shape)
print(b)
print(e)
```

张量支持加减乘除、矩阵运算（`matmul`）、索引切片等操作。只需调用 `.to("cuda")` 或 `.cuda()`，就能将张量移动到 GPU 上进行高效计算。

5.2.2 自动求导机制（autograd）

PyTorch 的 `autograd` 模块提供了强大的自动微分功能。只要在张量上开启梯度跟踪（`requires_grad=True`），PyTorch 就会构建计算图并在反向传播时自动计算梯度。

```
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

z = x**2 + y**3    # 构建计算图
z.backward()       # 反向传播

print(x.grad)    # dz/dx = 2x = 4
print(y.grad)    # dz/dy = 3y^2 = 27
```

5.2.3 构建与训练模型的基本流程

PyTorch 提供了 `nn.Module` 类作为所有模型的基类。训练流程通常包括：定义模型结构 → 定义损失函数和优化器 → 迭代训练（前向传播 → 计算损失 → 反向传播 → 参数更新）→ 验证评估。

```
import torch.nn as nn
import torch.optim as optim

# 定义模型
class MLP(nn.Module):
    def __init__(self, in_features, hidden, out_features):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(in_features, hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden, out_features)

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

model = MLP(784, 128, 10) # 例如 MNIST: 28*28=784
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# 训练一步
inputs = torch.randn(64, 784) # batch=64
labels = torch.randint(0, 10, (64,))
outputs = model(inputs)
loss = criterion(outputs, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print("训练一步的损失:", loss.item())
```

5.2.4 数据加载与处理（Dataset 与 DataLoader）

PyTorch 提供了 Dataset 与 DataLoader 两个核心组件：

Dataset 基类：需实现 `__len__()`（返回数据集大小）和 `__getitem__(index)`（返回索引对应的数据和标签）

DataLoader：用于批量加载数据，支持打乱（shuffle）、多进程（num_workers）、按需取样等

```
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([
```

```
transforms.ToTensor(),
transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = torchvision.datasets.MNIST(
    root="./data", train=True, transform=transform, download=True)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

for images, labels in train_loader:
    print(images.shape, labels.shape)
    break
```

5.2.5 小结

本节系统介绍了 PyTorch 的核心功能：张量操作是基础，autograd 提供了自动微分支持，nn.Module 与优化器定义了模型与训练流程，而 Dataset/DataLoader 负责高效加载与预处理数据。这些模块构成了 PyTorch 的核心逻辑，也是后续实现复杂深度学习模型（如 CNN、RNN、Transformer）的基石。

5.3 深度前馈网络

深度前馈网络（Deep Feedforward Network）是深度学习中最基础的一类神经网络模型。它通过将输入数据逐层传递，经由非线性激活函数的变换，最终得到输出结果。这类网络也被称为多层感知机（Multilayer Perceptron, MLP），在分类、回归等任务中都有着广泛应用。

5.3.1 感知机模型与激活函数

感知机（Perceptron）是神经网络最早的形式之一，由 Frank Rosenblatt 在 1958 年提出。感知机的基本思想是对输入向量 $x = [x_1, x_2, \dots, x_n]$ 进行加权求和，再加上偏置项 b ，最后通过一个激活函数 $f(\cdot)$ 得到输出：

$$y = f(\sum_i w_i x_i + b) = f(w^T x + b)$$

其中， $w = [w_1, w_2, \dots, w_n]$ 表示权重向量， b 为偏置项。

激活函数在神经网络中起到引入非线性的作用，使得网络能够逼近复杂的非线性函数。常见的激活函数包括：

1. Sigmoid 函数：

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

其输出范围在 $(0, 1)$ ，常用于二分类问题的输出层。但缺点是容易出现梯度消失。

2. 双曲正切函数（tanh）：

$$\tanh(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$$

输出范围在 $(-1, 1)$ ，相比 Sigmoid 更对称，收敛效果更好。

3. 修正线性单元 (ReLU):

$$f(x) = \max(0, x)$$

ReLU 是目前应用最广泛的激活函数，计算简单且有效缓解梯度消失问题。但也存在"神经元死亡"现象。

4. Leaky ReLU:

$$f(x) = x \quad (x \geq 0), \quad f(x) = \alpha x \quad (x < 0)$$

其中 α 是一个较小的常数，避免 ReLU 出现神经元死亡的问题。

```
import torch
```

```
import torch.nn as nn
```

```
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
```

```
sigmoid = nn.Sigmoid()
```

```
print("Sigmoid:", sigmoid(x))
```

```
tanh = nn.Tanh()
```

```
print("Tanh:", tanh(x))
```

```
relu = nn.ReLU()
```

```
print("ReLU:", relu(x))
```

```
leaky_relu = nn.LeakyReLU(0.1)
```

```
print("Leaky ReLU:", leaky_relu(x))
```

5.3.2 多层感知机 (MLP) 的结构与原理

单层感知机只能处理线性可分问题，而多层感知机 (MLP) 则通过叠加多层神经元结构，并在层与层之间加入非线性激活函数，从而能够逼近任意复杂的非线性函数。其基本结构为：

$$h^{(1)} = f(W^{(1)}x + b^{(1)})$$

$$h^{(2)} = f(W^{(2)}h^{(1)} + b^{(2)})$$

$$\hat{y} = f_{\text{out}}(W^{(L)}h^{(L-1)} + b^{(L)})$$

其中， f 是隐藏层激活函数， f_{out} 是输出层的激活函数（如 softmax 或 sigmoid）， $h^{(i)}$ 表示第 i 层的隐藏表示。

根据万能近似定理 (Universal Approximation Theorem)，只要隐藏层神经元数量足够，多层感知机能够以任意精度逼近任何连续函数。

5.3.3 前向传播与反向传播

1. 前向传播 (Forward Propagation)

前向传播是神经网络的计算过程：输入数据经过各层线性变换和非线性激活函数，逐层传递，直至得到输出预测结果。数学形式为：

$$\mathbf{a}^{(l)} = \mathbf{f}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

其中 $\mathbf{a}^{(l)}$ 表示第 l 层的激活值， $\mathbf{W}^{(l)}$ 、 $\mathbf{b}^{(l)}$ 分别为权重和偏置。

2. 反向传播 (Backpropagation)

反向传播的目的是计算损失函数 L 对每一层参数的梯度，以便使用梯度下降法进行参数更新。其核心是应用链式法则，将梯度从输出层逐步传播到输入层。

对于一个两层感知机，反向传播公式为：

$$\delta^{(2)} = (\partial L / \partial \hat{\mathbf{y}}) \odot \mathbf{g}'(\mathbf{z}^{(2)})$$

$$\partial L / \partial \mathbf{W}^{(2)} = \delta^{(2)} \mathbf{h}^T$$

$$\partial L / \partial \mathbf{b}^{(2)} = \delta^{(2)}$$

$$\delta^{(1)} = (\mathbf{W}^{(2)})^T \delta^{(2)} \odot \mathbf{f}'(\mathbf{z}^{(1)})$$

$$\partial L / \partial \mathbf{W}^{(1)} = \delta^{(1)} \mathbf{x}^T$$

$$\partial L / \partial \mathbf{b}^{(1)} = \delta^{(1)}$$

3. 手写推导 + 数值梯度对比验证

数值梯度近似公式 (finite difference method) 为：

$$\partial L / \partial \theta \approx [L(\theta + \epsilon) - L(\theta - \epsilon)] / (2\epsilon)$$

其中 ϵ 是一个很小的数 (如 10^{-5})。我们通过对比 PyTorch autograd 梯度、手写梯度与数值梯度近似来验证反向传播的正确性。

```
import torch

torch.manual_seed(42)

# 输入与标签
x = torch.tensor([[0.5, -1.0]], requires_grad=False)
y = torch.tensor([[1.0]], requires_grad=False)

# 初始化参数
W1 = torch.randn(2, 2, requires_grad=True)
b1 = torch.randn(2, requires_grad=True)
W2 = torch.randn(2, 1, requires_grad=True)
b2 = torch.randn(1, requires_grad=True)

# 前向传播
def forward(x, W1, b1, W2, b2):
    h = torch.relu(x @ W1 + b1)
```

```
        out = h @ W2 + b2
    return out, h

# 损失函数: MSE
def loss_fn(y_pred, y):
    return ((y_pred - y) ** 2).mean()

# 自动求导结果
y_pred, h = forward(x, W1, b1, W2, b2)
loss = loss_fn(y_pred, y)
loss.backward()

print("Autograd gradients:")
print("dL/dW1:", W1.grad)
print("dL/dW2:", W2.grad)
```

5.3.4 使用 PyTorch 构建简单前馈网络

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

# 数据预处理
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# 定义 MLP 模型
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(28*28, 256) # 输入层到隐藏层
        self.fc2 = nn.Linear(256, 128)   # 隐藏层到隐藏层
        self.fc3 = nn.Linear(128, 10)    # 输出层
        self.relu = nn.ReLU()
        self.softmax = nn.LogSoftmax(dim=1)
```

```

def forward(self, x):
    x = x.view(-1, 28*28) # 展平输入
    x = self.relu(self.fc1(x))
    x = self.relu(self.fc2(x))
    x = self.softmax(self.fc3(x))
    return x

# 实例化模型
model = MLP()
criterion = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练循环
for epoch in range(5):
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/5], Loss: {loss.item():.4f}')

```

5.3.5 小结

本节介绍了感知机与常见激活函数，解释了多层感知机的结构与原理，阐述了前向传播与反向传播机制，并通过 PyTorch 框架实现了一个简单的前馈神经网络。作为深度学习的入门模型，MLP 为理解更复杂的卷积神经网络（CNN）与循环神经网络（RNN）奠定了基础。

5.4 卷积神经网络（CNN）

卷积神经网络（Convolutional Neural Network, CNN）是深度学习领域中最经典的网络结构之一，尤其在计算机视觉任务（如图像分类、目标检测、语义分割等）中表现出色。与传统的全连接神经网络相比，CNN 通过局部连接、权值共享和空间降采样等机制，大幅减少了模型参数数量，同时提升了模型的泛化能力和计算效率。

5.4.1 卷积运算与特征提取的直观理解

1. 卷积的数学定义

给定输入图像 I 和卷积核 K ，二维离散卷积定义为：

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

其中， $S(i, j)$ 是输出特征图的位置 (i, j) 的值。

2. 卷积的直观理解

当卷积核是边缘检测滤波器（例如 Sobel、Laplacian），卷积输出能突出图像的边缘特征。

当卷积核是模糊滤波器，卷积结果会平滑图像，抑制噪声。

在深度学习中，卷积核参数不再固定，而是通过反向传播自动学习到适合特定任务的特征提取方式。

3. 卷积的关键超参数

卷积核大小（Kernel Size）：决定感受野范围，例如 3×3。

步幅（Stride）：卷积核每次滑动的距离，步幅越大，输出特征图越小。

填充（Padding）：在输入边缘补零，以保持输出大小。

5.4.2 池化层与特征图降维

池化层（Pooling Layer）的作用是降低特征图的空间维度，减少参数数量和计算量，同时增加模型的平移不变性。

最大池化（Max Pooling）：取区域内的最大值，常用于保留显著特征。

平均池化（Average Pooling）：取区域内的平均值，适合平滑特征。

5.4.3 常见卷积网络结构（LeNet、AlexNet、VGG 简介）

层次	LeNet-5	AlexNet	VGG16
输入层	32×32 灰度图	227×227 彩色图	224×224 彩色图
卷积层 1	5×5, 6 通道	11×11, 96 通道, stride=4	3×3, 64 通道
池化层 1	平均池化 2×2	最大池化 3×3, stride=2	最大池化 2×2
卷积层 2	5×5, 16 通道	5×5, 256 通道	3×3, 128 通道
全连接层	两层 FC (120, 84)	三层 FC (4096, 4096, 1000)	三层 FC (4096, 4096, 1000)
输出层	Softmax (10 类)	Softmax (1000 类)	Softmax (1000 类)

5.4.4 使用 PyTorch 构建简单的 CNN 模型

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# 定义 CNN 模型
class SimpleCNN(nn.Module):
    def __init__(self):
```

```
super(SimpleCNN, self).__init__()
self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
self.pool = nn.MaxPool2d(2, 2)
self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
self.fc1 = nn.Linear(64 * 8 * 8, 256)
self.fc2 = nn.Linear(256, 10)

def forward(self, x):
    x = self.pool(torch.relu(self.conv1(x)))
    x = self.pool(torch.relu(self.conv2(x)))
    x = x.view(-1, 64 * 8 * 8)
    x = torch.relu(self.fc1(x))
    x = self.fc2(x)
    return x

# 训练模型
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
net = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)

for epoch in range(5):
    running_loss = 0.0
    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    if i % 100 == 99:
        print(f'[{epoch+1}, {i+1}] loss: {running_loss / 100:.3f}')
        running_loss = 0.0
print("Finished Training")
```

5.5 循环神经网络（RNN）

现实中存在着大量的序列数据，例如文本（词序列）、语音（声波时间序列）、股价走势等。这类数据具有时间依赖性，当前时刻的数据往往依赖于之前的若干状态。为了解决此类问题，研究者提出了循环神经网络（Recurrent Neural Network, RNN）。

5.5.1 时间序列数据与循环结构的需求

时间序列数据的特点：数据具有顺序性；存在时间依赖性；数据可能具有长短期依赖。

传统前馈网络的局限性：MLP 和 CNN 无法直接捕获跨时间的依赖关系，它们将输入视为独立样本。

循环结构的需求：通过引入隐藏状态 h_t 来存储历史信息，使得网络在时刻 t 的计算能够利用 $t-1$ 的信息。

5.5.2 RNN 的基本结构与工作原理

循环神经网络的基本计算公式如下：

$$h_t = f(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

$$y_t = g(W_{yh} \cdot h_t + b_y)$$

其中： x_t 是时刻 t 的输入向量， h_t 是时刻 t 的隐藏状态（包含了历史信息）， y_t 是时刻 t 的输出， W_{xh} 、 W_{hh} 、 W_{yh} 是参数矩阵， f 、 g 是非线性激活函数（常用 $\tanh$ 或 ReLU ）。

5.5.3 长短期记忆网络（LSTM）与门控循环单元（GRU）简介

为解决标准 RNN 的梯度消失问题，研究者提出了改进的循环单元：LSTM（Long Short-Term Memory）和 GRU（Gated Recurrent Unit）。

LSTM 结构：LSTM 在隐藏状态外引入了细胞状态 c_t ，通过门机制来控制信息的流动。

三个主要的门：

遗忘门（控制哪些历史信息被遗忘）：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

输入门（控制新信息的写入）：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

输出门：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

最终的细胞状态更新：

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

GRU 结构：GRU 是 LSTM 的简化版本，通过两个门来控制：

更新门：

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

重置门：

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

隐藏状态更新:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

LSTM vs GRU 对比:

LSTM 结构更复杂, 参数更多, 适合长序列建模。

GRU 结构更简洁, 计算效率更高, 常用于实时应用。

实际应用中两者效果接近, 需结合具体任务选择。

5.5.4 使用 PyTorch 构建简单的 RNN 模型

```
import torch
import torch.nn as nn
import torch.optim as optim

# 定义模型
class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleRNN, self).__init__()
        self.hidden_size = hidden_size
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x, hidden=None):
        out, hidden = self.rnn(x, hidden)
        out = self.fc(out[:, -1, :]) # 取最后一个时间步
        return out, hidden

# 超参数
input_size = 10    # 输入维度
hidden_size = 20   # 隐藏层维度
output_size = 5    # 输出类别数

# 构建模型
model = SimpleRNN(input_size, hidden_size, output_size)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 伪造训练数据
X = torch.randn(100, 7, input_size) # batch=100, 序列长度=7
```

```
y = torch.randint(0, output_size, (100,))

# 训练
for epoch in range(50):
    optimizer.zero_grad()
    outputs, _ = model(X)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/50], Loss: {loss.item():.4f}')
```

5.5.5 小结

通过本节的学习，我们掌握了：序列数据与循环结构的需求；RNN 的基本计算公式与结构；LSTM 与 GRU 的门机制与对比；使用 PyTorch 构建简单 RNN 的实践方法。

5.6 实践（基于 Python、PyTorch）

本节通过一系列实践案例，综合使用多层感知机（MLP）、卷积神经网络（CNN）和循环神经网络（RNN），来完成典型的分类和序列建模任务。

5.6.1 数据预处理与加载（以 MNIST/CIFAR-10 为例）

```
import torch
import torchvision
import torchvision.transforms as transforms

# 定义数据预处理
transform_mnist = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

transform_cifar = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
# 加载 MNIST
train_mnist = torchvision.datasets.MNIST(
    root="./data", train=True, download=True, transform=transform_mnist)

# 加载 CIFAR-10
train_cifar = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True, transform=transform_cifar)

# 定义数据加载器
train_loader = torch.utils.data.DataLoader(train_mnist, batch_size=64, shuffle=True)
```

5.6.2 使用 MLP 完成图像分类任务

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(28*28, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28*28) # 展平
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

5.6.3 使用 CNN 进行图像识别

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64*8*8, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
```

```
x = torch.relu(self.conv2(x))
x = self.pool(x)
x = x.view(-1, 64*8*8)
x = torch.relu(self.fc1(x))
x = self.fc2(x)
return x
```

5.6.4 使用 RNN 完成文本序列建模任务

```
class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden_size, num_layers=1):
        super(CharRNN, self).__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.rnn = nn.RNN(hidden_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden=None):
        x = self.embed(x)
        out, hidden = self.rnn(x, hidden)
        out = self.fc(out)
        return out, hidden
```

5.6.5 模型训练过程的可视化与结果分析

为了更好地理解模型的训练效果，我们需要对损失函数和准确率的变化过程进行可视化：

```
import matplotlib.pyplot as plt

train_losses, val_losses = [], []
train_accs, val_accs = [], []

# 绘制曲线
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Curve")
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(train_accs, label="Train Acc")
plt.plot(val_accs, label="Val Acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy Curve")
plt.legend()

plt.show()
```

5.6.6 本节小结

掌握了 MNIST 与 CIFAR-10 数据预处理与加载方法。

理解了 MLP 在图像分类任务中的基本应用。

掌握了 CNN 在图像识别中的优势与实现方法。

了解了 RNN 在文本序列建模中的应用。

学会通过 loss 与 accuracy 曲线对训练过程进行分析与优化。